

Documentação de Consultas SQL

Sistema de Gerenciamento de Barbearia

Guia de Estudo para Apresentação

22 de novembro de 2025

Sumário

1	Introdução	5
1.1	Conceitos SQL Fundamentais	5
1.1.1	COALESCE	5
1.1.2	FILTER	5
1.1.3	CASE	5
2	Consultas de Estatísticas de Clientes	6
2.1	Listar Clientes com Estatísticas	6
2.1.1	Propósito	6
2.1.2	Consulta SQL	6
2.1.3	Explicação Detalhada	6
2.1.4	Conexão com a Rota	7
2.1.5	Uso no Frontend	7
3	Consultas de Detalhes do Cliente	8
3.1	Buscar Detalhes Completos de um Cliente	8
3.1.1	Propósito	8
3.1.2	Consulta SQL - Dados do Cliente	8
3.1.3	Consulta SQL - Histórico de Atendimentos	8
3.1.4	Explicação Detalhada	9
3.1.5	Conexão com a Rota	9
3.1.6	Uso no Frontend	9
4	Consultas de Agendamentos	10
4.1	Verificar Conflito de Horário	10
4.1.1	Propósito	10
4.1.2	Consulta SQL	10
4.1.3	Explicação Detalhada	10
4.1.4	Conexão com a Rota	11
4.1.5	Uso no Frontend	11
4.2	Obter Horários Disponíveis	11
4.2.1	Propósito	11
4.2.2	Lógica Implementada	11

4.2.3	Consulta SQL Base	11
4.2.4	Conexão com a Rota	12
4.2.5	Uso no Frontend	12
5	Consultas de Agendamentos do Cliente	13
5.1	Listar Agendamentos do Cliente Otimizado	13
5.1.1	Propósito	13
5.1.2	Consulta SQL	13
5.1.3	Explicação Detalhada	13
5.1.4	Conexão com a Rota	14
5.1.5	Uso no Frontend	14
6	Consultas de Avaliações	16
6.1	Listar Avaliações do Barbeiro com Paginação	16
6.1.1	Propósito	16
6.1.2	Consulta SQL	16
6.1.3	Explicação Detalhada	16
6.1.4	Conexão com a Rota	17
6.1.5	Uso no Frontend	17
7	Consultas de Produtos	19
7.1	Listar Produtos com Paginação e Filtros	19
7.1.1	Propósito	19
7.1.2	Consulta SQL	19
7.1.3	Explicação Detalhada	19
7.1.4	Conexão com a Rota	20
7.1.5	Uso no Frontend	20
7.2	Produtos com Estoque Baixo	20
7.2.1	Consulta SQL	20
7.2.2	Explicação	21
7.2.3	Uso	21
8	Consultas de Reservas	22
8.1	Listar Todas as Reservas com Informações Completas	22
8.1.1	Propósito	22
8.1.2	Consulta SQL	22
8.1.3	Explicação Detalhada	22
8.1.4	Conexão com a Rota	23
8.1.5	Uso no Frontend	23
8.2	Reservas por Status	23
8.2.1	Consulta SQL	23
8.2.2	Explicação	24
9	Consultas de Planos Mensais	25
9.1	Listar Todos os Planos com Serviços	25
9.1.1	Propósito	25
9.1.2	Consulta SQL	25
9.1.3	Explicação Detalhada	25
9.1.4	Conexão com a Rota	26

9.1.5	Uso no Frontend	27
9.2	Verificar Uso de Serviços do Plano	27
9.2.1	Propósito	27
9.2.2	Consulta SQL	27
9.2.3	Explicação Detalhada	28
9.2.4	Retorno Estruturado	28
9.2.5	Uso no Agendamento	29
10	Consultas de Relatórios Financeiros	30
10.1	Relatório Financeiro Completo	30
10.1.1	Propósito	30
10.1.2	Consulta SQL - Receitas	30
10.1.3	Explicação - Receitas	30
10.1.4	Consulta SQL - Gastos com Produtos	31
10.1.5	Explicação - Gastos	31
10.1.6	Consulta SQL - Vendas de Produtos	31
10.1.7	Explicação - Vendas	31
10.1.8	Consulta SQL - Planos	31
10.1.9	Explicação - Planos	32
10.1.10	Consulta SQL - Detalhamento por Serviço	32
10.1.11	Explicação - Detalhamento	32
10.1.12	Estrutura do Relatório Final	33
10.1.13	Conexão com a Rota	33
10.1.14	Uso no Frontend	33
11	Consultas de Notificações	35
11.1	Listar Clientes Inativos	35
11.1.1	Propósito	35
11.1.2	Consulta SQL	35
11.1.3	Explicação Detalhada	35
11.1.4	Conexão com a Rota	36
11.1.5	Uso no Frontend	37
11.2	Listar Clientes com Muitas Faltas	37
11.2.1	Consulta SQL	37
11.2.2	Explicação	38
11.2.3	Uso	38
12	Consultas de Horários e Disponibilidade	39
12.1	Verificação de Conflito de Horário	39
12.1.1	Algoritmo de Detecção de Conflito	39
12.1.2	Implementação	39
12.1.3	Uso no Agendamento	39
13	Boas Práticas e Otimizações	41
13.1	Uso de Índices	41
13.2	Evitar N+1 Queries	41
13.3	COALESCE para Valores Padrão	41
13.4	Paginação Eficiente	42

1 Introdução

Este documento apresenta uma análise detalhada das principais consultas SQL utilizadas no sistema de gerenciamento de barbearia. O objetivo é fornecer um guia de estudo que explique de forma clara e objetiva:

- O que cada consulta faz
- Como ela se conecta com as rotas da API
- Como o frontend utiliza essas informações
- Conceitos SQL importantes utilizados

1.1 Conceitos SQL Fundamentais

Antes de explorarmos as consultas, é importante entender três funções SQL essenciais:

1.1.1 COALESCE

Retorna o primeiro valor não-nulo de uma lista de expressões. É útil para substituir valores NULL por um padrão.

Exemplo: `COALESCE(total, 0)` retorna 0 se `total` for NULL.

1.1.2 FILTER

Permite aplicar uma condição em funções de agregação (como COUNT, SUM, AVG). Conta ou soma apenas os registros que atendem à condição.

Exemplo: `COUNT(*) FILTER (WHERE status = 'concluido')` conta apenas registros concluídos.

1.1.3 CASE

Estrutura condicional similar ao if-else. Permite executar diferentes lógicas baseadas em condições.

Exemplo:

```
1 CASE WHEN idade < 18 THEN 'Menor'
2     ELSE 'Maior' END
```

2 Consultas de Estatísticas de Clientes

2.1 Listar Clientes com Estatísticas

2.1.1 Propósito

Esta consulta retorna uma lista paginada de todos os clientes com estatísticas completas de atendimentos, incluindo total de cortes realizados, faltas, última visita e média de avaliações.

2.1.2 Consulta SQL

```
1 SELECT
2     c.cpf,
3     p.nome_completo,
4     p.email,
5     p.telefone,
6     p.data_nascimento,
7     COUNT(a.id_agendamento) FILTER (WHERE a.status = 'concluido')
8         as total_atendimentos,
9     COUNT(a.id_agendamento) FILTER (WHERE a.status = 'falta')
10        as total_faltas,
11     MAX(a.data_hora_agendamento) FILTER (WHERE a.status = '
12         concluido')
13         as ultima_visita,
14     COALESCE(AVG(av.nota), 0) as media_avaliacoes
15 FROM Cliente c
16 INNER JOIN Pessoa p ON c.cpf = p.cpf
17 LEFT JOIN Agendamento a ON c.cpf = a.client_id
18 LEFT JOIN Avaliacao av ON a.id_agendamento = av.id_agen
19 WHERE p.nome_completo ILIKE %s OR p.cpf LIKE %s
20 GROUP BY c.cpf, p.nome_completo, p.email, p.telefone, p.
21         data_nascimento
22 ORDER BY p.nome_completo
23 LIMIT %s OFFSET %s
```

2.1.3 Explicação Detalhada

1. **SELECT com agregações:** Seleciona dados básicos do cliente (CPF, nome, email) e calcula estatísticas usando funções de agregação.
2. **COUNT com FILTER:**
 - COUNT FILTER (WHERE status = 'concluido') conta apenas agendamentos concluídos
 - COUNT FILTER (WHERE status = 'falta') conta apenas as faltas registradas
3. **MAX com FILTER:** Retorna a data do último atendimento concluído.
4. **COALESCE na média:** Se não houver avaliações, retorna 0 ao invés de NULL.

5. JOINS utilizados:

- INNER JOIN com Pessoa: Todo cliente precisa ter dados pessoais
- LEFT JOIN com Agendamento: Cliente pode não ter agendamentos
- LEFT JOIN com Avaliacao: Agendamento pode não ter avaliação

6. **GROUP BY:** Agrupa por cliente para calcular as estatísticas individuais.

7. **Paginação:** LIMIT e OFFSET permitem carregar dados em partes.

2.1.4 Conexão com a Rota

Rota: GET /api/clientes/estatisticas

Controller: ClienteStatsController.listar_com_estatisticas()

Parâmetros aceitos:

- page: Número da página (padrão: 1)
- per_page: Itens por página (padrão: 10)
- search: Termo de busca por nome ou CPF

2.1.5 Uso no Frontend

Componente: GerenciarClientes.js

O frontend utiliza esta consulta para:

1. Exibir lista de clientes em cards com estatísticas
2. Implementar busca por nome ou CPF
3. Navegação entre páginas de resultados
4. Destacar clientes com muitas faltas ou inativos

Exemplo de uso:

```
1 const response = await api.get('/clientes/estatisticas', {  
2   params: { page, per_page: 10, search: termoBusca }  
3 });  
4 setClientes(response.data.clientes);
```

3 Consultas de Detalhes do Cliente

3.1 Buscar Detalhes Completos de um Cliente

3.1.1 Propósito

Retorna informações detalhadas de um cliente específico, incluindo estatísticas agregadas e histórico de atendimentos com avaliações.

3.1.2 Consulta SQL - Dados do Cliente

```
1 SELECT
2     c.cpf,
3     p.nome_completo,
4     p.email,
5     p.telefone,
6     p.endereco,
7     p.data_nascimento,
8     COUNT(a.id_agendamento) FILTER (WHERE a.status = 'concluido')
9         as total_atendimentos,
10    COUNT(a.id_agendamento) FILTER (WHERE a.status = 'falta')
11        as total_faltas,
12    MAX(a.data_hora_agendamento) FILTER (WHERE a.status = '
13        concluido')
14        as ultima_visita,
15    COALESCE(AVG(av.nota), 0) as media_avaliacoes,
16    COUNT(DISTINCT av.id_agen) as total_avaliacoes
17 FROM Cliente c
18 INNER JOIN Pessoa p ON c.cpf = p.cpf
19 LEFT JOIN Agendamento a ON c.cpf = a.client_id
20 LEFT JOIN Avaliacao av ON a.id_agendamento = av.id_agen
21 WHERE c.cpf = %s
22 GROUP BY c.cpf, p.nome_completo, p.email, p.telefone,
23          p.endereco, p.data_nascimento
```

3.1.3 Consulta SQL - Histórico de Atendimentos

```
1 SELECT
2     a.id_agendamento,
3     a.data_hora_agendamento,
4     a.status,
5     s.nome as servico_nome,
6     s.preco,
7     pb.nome_completo as barbeiro_nome,
8     av.nota,
9     av.comentario
10 FROM Agendamento a
11 INNER JOIN Contem ct ON a.id_agendamento = ct.id_agen
12 INNER JOIN Servico s ON ct.id_serv = s.id_servico
13 INNER JOIN Barbeiro b ON a.barbeiro_id = b.cpf
```



```

14 INNER JOIN Pessoa pb ON b.cpf = pb.cpf
15 LEFT JOIN Avaliacao av ON a.id_agendamento = av.id_agen
16 WHERE a.client_id = %s
17 ORDER BY a.data_hora_agendamento DESC
18 LIMIT 20

```

3.1.4 Explicação Detalhada

Primeira Query (Dados Agregados):

- Calcula estatísticas completas do cliente
- Usa COALESCE para evitar NULL na média de avaliações
- COUNT DISTINCT conta quantos agendamentos foram avaliados
- MAX retorna a data da última visita concluída

Segunda Query (Histórico):

- Busca os últimos 20 atendimentos do cliente
- Traz informações do serviço realizado e preço
- Inclui dados do barbeiro que realizou o atendimento
- LEFT JOIN com Avaliacao para incluir nota e comentário quando existir
- Ordena do mais recente para o mais antigo

3.1.5 Conexão com a Rota

Rota: GET /api/clientes/<cpf>/detalhes

Controller: ClienteStatsController.detalhes_cliente(cpf)

Retorno: JSON com dois objetos principais:

- **cliente:** Dados e estatísticas do cliente
- **historico:** Array com últimos atendimentos

3.1.6 Uso no Frontend

Componente: DetalhesCliente.js

Funcionalidades implementadas:

1. Exibe estatísticas do cliente em cards (atendimentos, faltas, média)
2. Mostra dados de contato completos
3. Lista histórico de atendimentos com avaliações
4. Permite realizar encaixe (agendamento direto pelo barbeiro)
5. Botão de voltar para lista de clientes

4 Consultas de Agendamentos

4.1 Verificar Conflito de Horário

4.1.1 Propósito

Verifica se há conflito de horário para um barbeiro ao tentar criar um novo agendamento, considerando a duração dos serviços.

4.1.2 Consulta SQL

```
1 SELECT a.*, s.duracao_estimada_min, pc.nome_completo as
   cliente_nome
2 FROM Agendamento a
3 JOIN Contem ct ON a.id_agendamento = ct.id_agen
4 JOIN Servico s ON ct.id_serv = s.id_servico
5 JOIN Cliente c ON a.client_id = c.cpf
6 JOIN Pessoa pc ON c.cpf = pc.cpf
7 WHERE a.barbeiro_id = %s
8 AND a.status IN ('pendente', 'confirmado')
9 AND DATE(a.data_hora_agendamento) = %s
10 AND a.id_agendamento != %s -- Opcional, para edi es
```

4.1.3 Explicação Detalhada

Lógica de verificação:

1. Busca todos os agendamentos ativos do barbeiro na data especificada
2. Para cada agendamento existente, calcula o intervalo ocupado:
 - Início: data_hora_agendamento
 - Fim: data_hora_agendamento + duracao_estimada_min
3. Compara com o novo agendamento proposto
4. Há conflito se os intervalos se sobrepõem

Filtros importantes:

- status IN ('pendente', 'confirmado'): Apenas agendamentos ativos
- DATE(...): Compara apenas a data, ignorando hora
- Exclui o próprio agendamento em caso de edição

4.1.4 Conexão com a Rota

Rota: GET /api/agendamentos/verificar-disponibilidade

Parâmetros:

- `cpf_barbeiro`: CPF do barbeiro
- `data_hora`: Data e hora do agendamento
- `duracao_min`: Duração estimada do serviço

Model: `Agendamento.verificar_conflito_horario()`

4.1.5 Uso no Frontend

Componente: `AgendarServico.js`

Utilizado para:

1. Validar horário antes de confirmar agendamento
2. Exibir mensagem de erro se houver conflito
3. Informar qual cliente está no horário conflitante

4.2 Obter Horários Disponíveis

4.2.1 Propósito

Retorna lista de horários disponíveis para um barbeiro em uma data específica, considerando duração do serviço.

4.2.2 Lógica Implementada

1. Busca todos agendamentos ativos do barbeiro na data
2. Define grade de horários possíveis (8h às 18h, intervalos de 30min)
3. Para cada horário da grade:
 - Calcula intervalo que seria ocupado pelo novo serviço
 - Verifica se há sobreposição com agendamentos existentes
 - Se não houver conflito, adiciona à lista de disponíveis
4. Retorna apenas horários livres

4.2.3 Consulta SQL Base

```
1 SELECT a.data_hora_agendamento, s.duracao_estimada_min
2 FROM Agendamento a
3 JOIN Contem ct ON a.id_agendamento = ct.id_agen
4 JOIN Servico s ON ct.id_serv = s.id_servico
5 WHERE a.barbeiro_id = %s
6 AND DATE(a.data_hora_agendamento) = %s
7 AND a.status IN ('pendente', 'confirmado')
8 ORDER BY a.data_hora_agendamento
```

4.2.4 Conexão com a Rota

Rota: GET /api/agendamentos/horarios-disponiveis

Parâmetros:

- `cpf_barbeiro`: CPF do barbeiro
- `data`: Data no formato YYYY-MM-DD
- `duracao_servico_min`: Duração do serviço em minutos

Retorno:

```
1 {  
2   "data": "2025-01-15",  
3   "cpf_barbeiro": "12345678900",  
4   "duracao_servico_min": 30,  
5   "horarios_disponiveis": ["08:00", "08:30", "10:00", ...],  
6   "total_horarios": 15  
7 }
```

4.2.5 Uso no Frontend

Componente: AgendarServico.js

Interface implementada:

1. Cliente seleciona barbeiro, serviço e data
2. Sistema carrega horários disponíveis automaticamente
3. Exibe horários em grupos (manhã, tarde, noite)
4. Cliente clica no horário desejado
5. Mostra resumo do agendamento antes de confirmar

5 Consultas de Agendamentos do Cliente

5.1 Listar Agendamentos do Cliente Otimizado

5.1.1 Propósito

Retorna todos os agendamentos de um cliente com informações completas em uma única consulta otimizada, incluindo dados do serviço, barbeiro e avaliação.

5.1.2 Consulta SQL

```
1 SELECT
2     a.id_agendamento ,
3     a.data_hora_agendamento ,
4     a.status ,
5     a.cpf_origem ,
6     a.client_id ,
7     a.barbeiro_id ,
8     s.id_servico ,
9     s.nome as servico_nome ,
10    s.preco ,
11    s.duracao_estimada_min ,
12    s.descricao as servico_descricao ,
13    pb.nome_completo as barbeiro_nome ,
14    pb.telefone as barbeiro_telefone ,
15    pb.email as barbeiro_email ,
16    av.nota as avaliacao_nota ,
17    av.comentario as avaliacao_comentario ,
18    CASE WHEN av.id_agen IS NOT NULL THEN true
19         ELSE false END as tem_avaliacao
20 FROM Agendamento a
21 INNER JOIN Contem ct ON a.id_agendamento = ct.id_agen
22 INNER JOIN Servico s ON ct.id_serv = s.id_servico
23 INNER JOIN Barbeiro b ON a.barbeiro_id = b.cpf
24 INNER JOIN Pessoa pb ON b.cpf = pb.cpf
25 LEFT JOIN Avaliacao av ON a.id_agendamento = av.id_agen
26 WHERE a.client_id = %s
27 ORDER BY a.data_hora_agendamento DESC
```

5.1.3 Explicação Detalhada

Por que é otimizada?

- Traz TODAS as informações necessárias em UMA consulta
- Evita N+1 queries (consultas repetidas para cada agendamento)
- Usa INNER JOIN para dados obrigatórios (serviço, barbeiro)
- Usa LEFT JOIN para dados opcionais (avaliação)

CASE para flag booleana:

```
1 CASE WHEN av.id_agen IS NOT NULL THEN true
2     ELSE false END as tem_avaliacao
```

Cria campo boolean que indica se o agendamento já foi avaliado, facilitando validações no frontend.

Ordenação:

- ORDER BY DESC: Agendamentos mais recentes primeiro
- Útil para mostrar próximos agendamentos no topo

5.1.4 Conexão com a Rota

Rota: GET /api/clientes/me/agendamentos-otimizado

Controller: AgendamentoController.listar_agendamentos_cliente_otimizado()

Model: Agendamento.buscar_agendamentos_cliente_otimizado(cpf)

5.1.5 Uso no Frontend

Componente: MeusAgendamentos.js

Funcionalidades implementadas:

1. **Listagem completa** com todas as informações em cards
2. **Filtros:**
 - Por status (todos, pendentes, concluídos, cancelados)
 - Busca por nome de serviço ou barbeiro
 - Ordenação por data (recentes/antigas)
3. **Paginação:** 6 agendamentos por página
4. **Ações condicionais:**
 - Cancelar (se faltar mais de 2h)
 - Avaliar (se concluído e não avaliado)
 - Exibe se já foi avaliado
5. **Estatísticas:** Total, pendentes, concluídos

Exemplo de uso:

```
1 // Carrega agendamentos
2 const response = await api.get('/clientes/me/agendamentos-
  otimizado');
3 const agendamentos = response.data;
4
5 // Filtra por status
6 const pendentes = agendamentos.filter(a =>
7     a.status === 'pendente' || a.status === 'confirmado'
8 );
9
```

```
10 // Verifica se pode avaliar
11 const podeAvaliar = agendamento.status === 'concluido'
12    && !agendamento.tem_avaliacao;
```

6 Consultas de Avaliações

6.1 Listar Avaliações do Barbeiro com Paginação

6.1.1 Propósito

Retorna avaliações recebidas por um barbeiro com paginação e filtros opcionais por data e nota mínima.

6.1.2 Consulta SQL

```
1  -- Contar total de avaliacoes
2  SELECT COUNT(*) as total
3  FROM Avaliacao av
4  JOIN Agendamento a ON av.id_agen = a.id_agendamento
5  JOIN Cliente c ON a.client_id = c.cpf
6  JOIN Pessoa pc ON c.cpf = pc.cpf
7  JOIN Contem ct ON a.id_agendamento = ct.id_agen
8  JOIN Servico s ON ct.id_serv = s.id_servico
9  WHERE a.barbeiro_id = %s
10     AND DATE(a.data_hora_agendamento) >= %s -- Filtro opcional
11     AND DATE(a.data_hora_agendamento) <= %s -- Filtro opcional
12     AND av.nota >= %s -- Filtro opcional
13
14  -- Buscar avaliacoes paginadas
15  SELECT
16      av.*,
17      a.data_hora_agendamento,
18      pc.nome_completo as cliente_nome,
19      s.nome as servico_nome
20  FROM Avaliacao av
21  JOIN Agendamento a ON av.id_agen = a.id_agendamento
22  JOIN Cliente c ON a.client_id = c.cpf
23  JOIN Pessoa pc ON c.cpf = pc.cpf
24  JOIN Contem ct ON a.id_agendamento = ct.id_agen
25  JOIN Servico s ON ct.id_serv = s.id_servico
26  WHERE a.barbeiro_id = %s
27      [FILTROS OPCIONAIS]
28  ORDER BY a.data_hora_agendamento DESC
29  LIMIT %s OFFSET %s
```

6.1.3 Explicação Detalhada

Duas consultas necessárias:

1. COUNT para saber total de registros (paginação)
2. SELECT com LIMIT/OFFSET para buscar página específica

Filtros dinâmicos:

- data_inicio e data_fim: Período de avaliações

- `nota_min`: Mostra apenas avaliações acima de X estrelas
- Todos são opcionais

JOINS encadeados:

```
1 Avaliacao -> Agendamento -> Cliente -> Pessoa
2                                     -> Contem -> Servico
```

Permite trazer nome do cliente e serviço avaliado.

6.1.4 Conexão com a Rota

Rota: GET /api/avaliacoes/me

Controller: `AgendamentoController.listar_avaliacoes_paginado()`

Parâmetros query string:

- `page`: Número da página (padrão: 1)
- `per_page`: Itens por página (padrão: 10)
- `data_inicio`: Data inicial (opcional)
- `data_fim`: Data final (opcional)
- `nota_min`: Nota mínima (opcional)

Retorno:

```
1 {
2   "avaliacoes": [...],
3   "total_avaliacoes": 45,
4   "total_paginas": 5,
5   "pagina_atual": 1,
6   "por_pagina": 10,
7   "tem_proxima": true,
8   "tem_anterior": false
9 }
```

6.1.5 Uso no Frontend

Componente: `MinhasAvaliacoes.js`

Interface completa com:

1. Cards mostrando:
 - Estrelas da avaliação (1-5)
 - Nome do cliente
 - Serviço avaliado
 - Data do atendimento
 - Comentário (se houver)
2. Filtros por período

3. Navegação entre páginas
4. Estatísticas gerais (média, total)
5. Distribuição de notas (quantas 5 estrelas, 4 estrelas, etc)

7 Consultas de Produtos

7.1 Listar Produtos com Paginação e Filtros

7.1.1 Propósito

Retorna lista paginada de produtos com filtros por nome, categoria e status, permitindo busca e navegação eficientes.

7.1.2 Consulta SQL

```
1  -- Contar total
2  SELECT COUNT(*) as total
3  FROM Produto
4  WHERE 1=1
5         AND LOWER(nome_produto) LIKE LOWER(%s)  -- Filtro opcional
6         AND categoria = %s                        -- Filtro opcional
7         AND status = %s                          -- Filtro opcional
8
9  -- Buscar produtos paginados
10 SELECT *
11 FROM Produto
12 WHERE 1=1
13        [FILTROS OPCIONAIS]
14 ORDER BY nome_produto
15 LIMIT %s OFFSET %s
```

7.1.3 Explicação Detalhada

WHERE 1=1:

- Técnica para facilitar adição dinâmica de filtros
- Sempre verdadeiro, permite adicionar AND posteriormente
- Simplifica código quando filtros são opcionais

LOWER e LIKE:

```
1 LOWER(nome_produto) LIKE LOWER(%s)
```

- LOWER: Ignora maiúsculas/minúsculas na busca
- LIKE com %: Busca parcial (contém o termo)
- Exemplo: "pomada" encontra "Pomada Fixadora Premium"

Paginação:

- LIMIT: Quantos registros retornar
- OFFSET: Quantos pular (calculado: (página-1) * por_página)

7.1.4 Conexão com a Rota

Rota: GET /api/produtos/paginado

Controller: ProdutoController.listar_paginado()

Parâmetros:

- pagina: Número da página
- por_pagina: Itens por página (1-100)
- nome: Busca por nome (opcional)
- categoria: Filtrar por categoria (opcional)
- status: Filtrar por status (opcional)

7.1.5 Uso no Frontend

Componente: ListarProdutos.js

Funcionalidades:

1. Barra de busca por nome
2. Filtros por categoria e status
3. Seletor de itens por página (5, 10, 20, 50)
4. Navegação com botões:
 - Primeira página (◀◀)
 - Anterior
 - Números das páginas
 - Próxima
 - Última página (≫≫)
5. Contador: "Mostrando X-Y de Z produtos"
6. Cards com informações do produto
7. Ações: Ajustar estoque, Editar, Deletar

7.2 Produtos com Estoque Baixo

7.2.1 Consulta SQL

```
1 SELECT * FROM Produto
2 WHERE quantidade_estoque <= minimo_estoque
3 ORDER BY quantidade_estoque
```

7.2.2 Explicação

- Compara estoque atual com estoque mínimo definido
- Ordena por quantidade (mais críticos primeiro)
- Usado para alertas no dashboard

7.2.3 Uso

Exibido no dashboard do barbeiro chefe para reposição.

8 Consultas de Reservas

8.1 Listar Todas as Reservas com Informações Completas

8.1.1 Propósito

Retorna todas as reservas de produtos com informações completas do cliente e produto em uma única consulta.

8.1.2 Consulta SQL

```
1 SELECT
2     r.id_cliente ,
3     r.id_prod ,
4     r.data_reserva ,
5     r.status ,
6     p.nome_completo as cliente_nome ,
7     p.email as cliente_email ,
8     p.telefone as cliente_telefone ,
9     prod.nome_produto ,
10    prod.categoria ,
11    prod.preco_venda ,
12    prod.quantidade_estoque
13 FROM Reserva r
14 JOIN Cliente c ON r.id_cliente = c.cpf
15 JOIN Pessoa p ON c.cpf = p.cpf
16 JOIN Produto prod ON r.id_prod = prod.id_produto
17 ORDER BY r.data_reserva DESC
```

8.1.3 Explicação Detalhada

JOINS múltiplos:

```
1 Reserva -> Cliente -> Pessoa (dados do cliente)
2         -> Produto (dados do produto)
```

Informações trazidas:

- Dados da reserva (data, status)
- Dados do cliente (nome, email, telefone)
- Dados do produto (nome, categoria, preço, estoque)

Ordenação:

- DESC: Reservas mais recentes primeiro
- Útil para acompanhamento de novas reservas

8.1.4 Conexão com a Rota

Rota: GET /api/reservas/todas

Restrição: Apenas barbeiros (decorador @barbeiro_required)

Controller: ReservaController.listar_todas()

8.1.5 Uso no Frontend

Componente: ConsultarReservas.js

Interface completa com:

1. Estatísticas em cards:
 - Total de reservas
 - Reservas pendentes
 - Compradas/Retiradas
 - Canceladas
2. Filtros múltiplos:
 - Por status
 - Por categoria de produto
 - Por período (data início/fim)
 - Por nome do cliente
3. Cards de reserva mostrando:
 - Nome do cliente e data
 - Produto e categoria
 - Preço e estoque disponível
 - Dados de contato do cliente
4. Ações:
 - Alterar status
 - Cancelar reserva
 - Ver detalhes

8.2 Reservas por Status

8.2.1 Consulta SQL

```
1 SELECT
2     [mesmos campos da consulta anterior]
3 FROM Reserva r
4 JOIN Cliente c ON r.id_cliente = c.cpf
5 JOIN Pessoa p ON c.cpf = p.cpf
6 JOIN Produto prod ON r.id_prod = prod.id_produto
7 WHERE r.status = %s
8 ORDER BY r.data_reserva DESC
```

8.2.2 Explicação

Mesma estrutura da query anterior, mas com filtro WHERE adicional para status específico.

Status possíveis:

- reservado: Cliente reservou, aguardando retirada
- comprado: Cliente comprou/retirou o produto
- retirado: Confirmação de retirada
- cancelado: Reserva cancelada
- pendente: Status temporário

9 Consultas de Planos Mensais

9.1 Listar Todos os Planos com Serviços

9.1.1 Propósito

Retorna todos os planos mensais cadastrados com seus serviços inclusos, descontos e valores calculados.

9.1.2 Consulta SQL

```
1 SELECT
2     pm.id_plano_mensal,
3     pm.id_barbeiro_chefe,
4     p.nome_completo as criador_nome,
5     s.id_servico,
6     s.nome as servico_nome,
7     s.preco as servico_preco,
8     ps.quantidade as servico_quantidade,
9     ps.desconto as servico_desconto
10 FROM Plano_Mensal pm
11 JOIN Barbeiro_Chefe bc ON pm.id_barbeiro_chefe = bc.
12     id_barbeiro_chefe
13 JOIN Pessoa p ON bc.cpf_barbeiro = p.cpf
14 LEFT JOIN Possui ps ON pm.id_plano_mensal = ps.id_plano
15 LEFT JOIN Servico s ON ps.id_serv = s.id_servico
16 ORDER BY pm.id_plano_mensal DESC, s.nome
```

9.1.3 Explicação Detalhada

LEFT JOIN importante:

- Usa LEFT JOIN com Possui e Servico
- Permite retornar planos mesmo sem serviços cadastrados
- Cada linha representa um serviço do plano
- Um plano com 3 serviços retorna 3 linhas

Processamento pós-consulta: O código Python/JavaScript agrupa as linhas:

```
1 # Agrupar servicos por plano
2 planos_dict = {}
3 for row in resultados:
4     plano_id = row['id_plano_mensal']
5     if plano_id not in planos_dict:
6         planos_dict[plano_id] = {
7             'id_plano_mensal': plano_id,
8             'criador_nome': row['criador_nome'],
9             'servicos': []
10         }
```

```

11     if row['id_servico']:
12         planos_dict[plano_id]['servicos'].append({
13             'id_servico': row['id_servico'],
14             'nome': row['servico_nome'],
15             'preco': row['servico_preco'],
16             'quantidade': row['servico_quantidade'],
17             'desconto': row['servico_desconto']
18         })

```

Cálculo de valores: Para cada plano, calcula-se:

```

1  valor_sem_desconto = 0
2  valor_com_desconto = 0
3
4  for servico in plano['servicos']:
5      subtotal = preco * quantidade
6      valor_desconto = subtotal * (desconto / 100)
7      valor_com_desc = subtotal - valor_desconto
8
9      valor_sem_desconto += subtotal
10     valor_com_desconto += valor_com_desc
11
12     economia = valor_sem_desconto - valor_com_desconto
13     desconto_medio = (economia / valor_sem_desconto) * 100

```

9.1.4 Conexão com a Rota

Rota: GET /api/planos

Controller: PlanoController.listar()

Model: PlanoMensal.listar_todos()

Retorno estruturado:

```

1  [
2      {
3          "id_plano_mensal": 1,
4          "criador_nome": "João Barbeiro",
5          "servicos": [
6              {
7                  "id_servico": 1,
8                  "nome": "Corte Masculino",
9                  "preco": 50.00,
10                 "quantidade": 2,
11                 "desconto": 15.0
12             },
13             ...
14         ],
15         "valor_sem_desconto": 100.00,
16         "valor_com_desconto": 85.00,
17         "valor_desconto_total": 15.00,
18         "desconto_medio": 15.0
19     },
20     ...

```

9.1.5 Uso no Frontend

Componente Cliente: MeusPlanos.js

- Exibe planos disponíveis para assinatura
- Mostra cálculo de economia
- Destaca desconto médio do plano
- Botão para assinar plano

Componente Barbeiro: GerenciarPlanos.js

- Lista todos os planos criados
- Permite editar serviços e descontos
- Opção de deletar plano
- Busca por ID, criador ou serviço

9.2 Verificar Uso de Serviços do Plano

9.2.1 Propósito

Verifica quantos serviços de um plano o cliente já utilizou no período da assinatura.

9.2.2 Consulta SQL

```
1  -- Buscar assinatura ativa
2  SELECT data_inicio, data_fim
3  FROM Assina
4  WHERE id_cliente = %s AND id_plano = %s
5  AND data_fim >= CURRENT_DATE
6  ORDER BY data_fim DESC
7  LIMIT 1
8
9  -- Buscar servicos do plano
10 SELECT ps.id_serv, ps.quantidade, ps.desconto, s.nome, s.preco
11 FROM Possui ps
12 JOIN Servico s ON ps.id_serv = s.id_servico
13 WHERE ps.id_plano = %s
14
15 -- Para cada servico, contar uso
16 SELECT COUNT(*) as usado
17 FROM Agendamento a
18 JOIN Contem c ON a.id_agendamento = c.id_agen
19 WHERE a.client_id = %s
20 AND c.id_serv = %s
21 AND a.data_hora_agendamento >= %s
```

```
22 AND a.data_hora_agendamento <= %s
23 AND a.status IN ('concluido', 'confirmado', 'pendente')
```

9.2.3 Explicação Detalhada

Três consultas necessárias:

1. Buscar período da assinatura ativa
2. Buscar todos os serviços incluídos no plano
3. Para cada serviço, contar quantas vezes foi usado

Contagem de uso:

- Conta agendamentos no período da assinatura
- Considera status: concluído, confirmado, pendente
- Não conta cancelados ou faltas

Cálculo de disponibilidade:

```
1 quantidade_disponivel = quantidade_plano - quantidade_usada
2 esgotado = (quantidade_usada >= quantidade_plano)
```

9.2.4 Retorno Estruturado

```
1 {
2     "id_plano": 1,
3     "data_inicio": "2025-01-01",
4     "data_fim": "2025-01-31",
5     "servicos": [
6         {
7             "id_servico": 1,
8             "nome_servico": "Corte Masculino",
9             "quantidade_plano": 2,
10            "quantidade_usada": 1,
11            "quantidade_disponivel": 1,
12            "esgotado": false,
13            "desconto_percentual": 15.0
14        },
15        ...
16    ]
17 }
```

9.2.5 Uso no Agendamento

Fluxo ao agendar:

1. Cliente escolhe serviço
2. Sistema verifica se tem plano ativo com esse serviço
3. Se sim, mostra opção de usar o plano
4. Valida se ainda tem quantidade disponível
5. Se disponível, permite agendar sem custo
6. Atualiza contador de uso

No frontend (AgendarServico.js):

```
1 // Verifica se pode usar plano
2 const response = await api.get(`/planos/pode-agendar/${idServico
  }`);
3
4 if (response.data.pode_usar_plano) {
5   // Mostra checkbox para usar plano
6   // Exibe informa es de uso
7   setPodeUsarPlano(true);
8   setInfoPlano(response.data);
9 }
10
11 // Ao confirmar com plano
12 await api.post(`/agendamentos`, {
13   ...dados,
14   usar_plano: true
15 });
```

10 Consultas de Relatórios Financeiros

10.1 Relatório Financeiro Completo

10.1.1 Propósito

Gera relatório financeiro detalhado com receitas, gastos, lucros e métricas de atendimento para um período específico.

10.1.2 Consulta SQL - Receitas

```
1 SELECT
2     COALESCE(SUM(s.preco), 0) as total_receitas,
3     COUNT(DISTINCT a.id_agendamento) as total_atendimentos,
4     COUNT(DISTINCT a.client_id) as clientes_atendidos,
5     COALESCE(AVG(s.preco), 0) as ticket_medio
6 FROM Agendamento a
7 JOIN Contem c ON a.id_agendamento = c.id_agen
8 JOIN Servico s ON c.id_serv = s.id_servico
9 WHERE a.status = 'concluido'
10 AND DATE(a.data_hora_agendamento) BETWEEN %s AND %s
11 AND a.barbeiro_id = %s -- Opcional
```

10.1.3 Explicação - Receitas

COALESCE em agregações:

- Evita NULL quando não há dados no período
- Retorna 0 ao invés de NULL
- Facilita cálculos posteriores

Métricas calculadas:

1. total_receitas: Soma de todos os preços (SUM)
2. total_atendimentos: Conta agendamentos únicos (COUNT DISTINCT)
3. clientes_atendidos: Conta clientes únicos
4. ticket_medio: Valor médio por atendimento (AVG)

Filtros aplicados:

- Apenas status 'concluido' (serviços efetivamente prestados)
- Período entre datas especificadas
- Opcionalmente filtra por barbeiro específico

10.1.4 Consulta SQL - Gastos com Produtos

```
1 SELECT
2     COALESCE(SUM(p.preco_compra), 0) as total_gastos_produtos,
3     COUNT(*) as total_produtos_comprados
4 FROM Reserva r
5 JOIN Produto p ON r.id_prod = p.id_produto
6 WHERE r.status IN ('comprado', 'retirado')
7 AND DATE(r.data_reserva) BETWEEN %s AND %s
```

10.1.5 Explicação - Gastos

- Soma preços de COMPRA (custo para a barbearia)
- Considera apenas produtos efetivamente vendidos
- Status 'comprado' ou 'retirado' indicam venda concretizada

10.1.6 Consulta SQL - Vendas de Produtos

```
1 SELECT
2     COALESCE(SUM(p.preco_venda), 0) as receita_produtos,
3     COALESCE(SUM(p.preco_venda - p.preco_compra), 0) as
4     lucro_produtos,
5     COUNT(*) as total_vendas
6 FROM Reserva r
7 JOIN Produto p ON r.id_prod = p.id_produto
8 WHERE r.status IN ('comprado', 'retirado')
9 AND DATE(r.data_reserva) BETWEEN %s AND %s
```

10.1.7 Explicação - Vendas

Cálculo de lucro direto no SQL:

```
1 SUM(preco_venda - preco_compra) as lucro_produtos
```

- Calcula margem de cada produto vendido
- Soma todas as margens
- Resultado: lucro total com produtos

10.1.8 Consulta SQL - Planos

```
1 SELECT
2     COUNT(*) as total_assinaturas,
3     COUNT(*) FILTER (WHERE DATE(data_inicio) BETWEEN %s AND %s)
4     as novas_assinaturas,
5     COUNT(*) FILTER (WHERE DATE(data_fim) BETWEEN %s AND %s
6     AND data_fim < CURRENT_DATE) as cancelamentos
7 FROM Assina
8 WHERE data_fim >= CURRENT_DATE OR data_fim IS NULL
```

10.1.9 Explicação - Planos

FILTER com diferentes condições:

1. total_assinaturas: Conta todas as ativas
2. novas_assinaturas: FILTER onde data_inicio está no período
3. cancelamentos: FILTER onde data_fim está no período E já venceu

Diferença de WHERE vs FILTER:

- WHERE: Filtra linhas ANTES de agregar
- FILTER: Aplica condição DENTRO da agregação
- Permite múltiplas agregações com condições diferentes

10.1.10 Consulta SQL - Detalhamento por Serviço

```
1 SELECT
2     s.nome as servico_nome,
3     COUNT(*) as quantidade,
4     COALESCE(SUM(s.preco), 0) as receita_total,
5     COALESCE(AVG(s.preco), 0) as preco_medio
6 FROM Agendamento a
7 JOIN Contem c ON a.id_agendamento = c.id_agen
8 JOIN Servico s ON c.id_serv = s.id_servico
9 WHERE a.status = 'concluido'
10 AND DATE(a.data_hora_agendamento) BETWEEN %s AND %s
11 GROUP BY s.id_servico, s.nome
12 ORDER BY receita_total DESC
```

10.1.11 Explicação - Detalhamento

GROUP BY com múltiplas colunas:

- Agrupa por serviço (id e nome)
- Para cada serviço, calcula estatísticas
- Permite identificar serviços mais rentáveis

ORDER BY em agregação:

- Ordena por receita_total decendente
- Mostra serviços mais lucrativos primeiro
- Útil para análise de produtos estrela

10.1.12 Estrutura do Relatório Final

```
1 {
2   "periodo": {
3     "data_inicio": "2025-01-01",
4     "data_fim": "2025-01-31"
5   },
6   "resumo_financeiro": {
7     "receita_total": 15000.00,
8     "receita_servicos": 12000.00,
9     "receita_produtos": 2500.00,
10    "receita_planos": 500.00,
11    "gastos_total": 3000.00,
12    "lucro_liquido": 12000.00,
13    "margem_lucro": 80.00
14  },
15  "metricas_atendimento": {
16    "total_atendimentos": 240,
17    "clientes_atendidos": 180,
18    "ticket_medio": 50.00
19  },
20  "produtos": { ... },
21  "planos": { ... },
22  "detalhamento_servicos": [ ... ]
23 }
```

10.1.13 Conexão com a Rota

Rota: GET /api/relatorios/financeiro

Parâmetros obrigatórios:

- data_inicio: Data inicial (YYYY-MM-DD)
- data_fim: Data final (YYYY-MM-DD)

Parâmetros opcionais:

- cpf_barbeiro: Filtrar por barbeiro específico

Controller: RelatorioController.gerar_relatorio_financeiro()

10.1.14 Uso no Frontend

Componente: RelatoriosFinanceiros.js (presumido)

Funcionalidades:

1. Seleção de período (data início/fim)
2. Cards com resumo financeiro
3. Gráficos de receitas por fonte
4. Tabela de serviços mais rentáveis

- 5. Métricas de atendimento
- 6. Indicadores de crescimento
- 7. Exportação para PDF/Excel

11 Consultas de Notificações

11.1 Listar Clientes Inativos

11.1.1 Propósito

Identifica clientes que não realizaram cortes há muito tempo para campanhas de reativação.

11.1.2 Consulta SQL

```
1 WITH ultimo_atendimento AS (  
2     SELECT  
3         a.client_id,  
4         MAX(a.data_hora_agendamento) as ultima_visita  
5     FROM Agendamento a  
6     WHERE a.barbeiro_id = %s  
7     AND a.status = 'concluido'  
8     GROUP BY a.client_id  
9 )  
10 SELECT  
11     c.cpf,  
12     p.nome_completo,  
13     p.email,  
14     p.telefone,  
15     ua.ultima_visita,  
16     CURRENT_DATE - DATE(ua.ultima_visita) as dias_sem_visita,  
17     COUNT(DISTINCT a.id_agendamento) FILTER (WHERE a.status = '  
18         concluido')  
19     as total_atendimentos  
20 FROM ultimo_atendimento ua  
21 JOIN Cliente c ON ua.client_id = c.cpf  
22 JOIN Pessoa p ON c.cpf = p.cpf  
23 LEFT JOIN Agendamento a ON c.cpf = a.client_id  
24     AND a.barbeiro_id = %s  
25 WHERE CURRENT_DATE - DATE(ua.ultima_visita) >= %s  
26 GROUP BY c.cpf, p.nome_completo, p.email, p.telefone, ua.  
ultima_visita  
ORDER BY dias_sem_visita DESC
```

11.1.3 Explicação Detalhada

CTE (Common Table Expression) - WITH:

```
1 WITH ultimo_atendimento AS (  
2     SELECT client_id, MAX(data_hora_agendamento) as ultima_visita  
3     FROM Agendamento  
4     WHERE status = 'concluido'  
5     GROUP BY client_id  
6 )
```

- Cria tabela temporária chamada `ultimo_atendimento`
- Calcula data do último corte concluído de cada cliente
- Simplifica query principal
- Melhora legibilidade

Cálculo de dias sem visita:

```
1 CURRENT_DATE - DATE(ultima_visita) as dias_sem_visita
```

- Subtrai datas para obter diferença em dias
- `CURRENT_DATE`: Data atual
- `DATE()`: Converte timestamp para data
- Resultado: número inteiro de dias

Filtro de inatividade:

```
1 WHERE CURRENT_DATE - DATE(ua.ultima_visita) >= %s
```

- Parâmetro configurável (ex: 60 dias)
- Retorna apenas clientes além do limite

11.1.4 Conexão com a Rota

Rota: GET `/api/notificacoes/clientes-inativos`

Parâmetros:

- `dias`: Mínimo de dias sem visita (padrão: 60)

Controller: `NotificacaoController.listar_clientes_inativos()`

Retorno:

```
1 {
2   "clientes": [
3     {
4       "cpf": "12345678900",
5       "nome_completo": "João Silva",
6       "email": "joao@email.com",
7       "telefone": "(11) 98888-0000",
8       "ultima_visita": "2024-09-15",
9       "dias_sem_visita": 98,
10      "total_atendimentos": 15
11    },
12    ...
13  ],
14  "total": 25,
15  "dias_filtro": 60
16 }
```

11.1.5 Uso no Frontend

Componente: NotificarClientes.js

Fluxo de uso:

1. Barbeiro seleciona "Clientes Inativos"
2. Define período de inatividade (ex: 60 dias)
3. Sistema busca clientes que não cortam há 60+ dias
4. Exibe lista com informações de cada cliente
5. Barbeiro seleciona clientes para notificar
6. Escolhe template de email (ex: "Sentimos sua falta")
7. Personaliza mensagem (opcional)
8. Envia emails em lote

Interface:

- Cards com dados do cliente
- Checkbox para seleção
- Botão "Selecionar Todos"
- Contador de selecionados
- Preview do email antes de enviar

11.2 Listar Clientes com Muitas Faltas

11.2.1 Consulta SQL

```
1 SELECT
2     c.cpf,
3     p.nome_completo,
4     p.email,
5     p.telefone,
6     COUNT(*) FILTER (WHERE a.status = 'falta') as total_faltas,
7     COUNT(*) FILTER (WHERE a.status = 'concluido') as
8         total_atendimentos,
9     MAX(a.data_hora_agendamento) FILTER (WHERE a.status = 'falta'
10         )
11         as ultima_falta
12 FROM Cliente c
13 JOIN Pessoa p ON c.cpf = p.cpf
14 JOIN Agendamento a ON c.cpf = a.client_id
15 WHERE a.barbeiro_id = %s
16 GROUP BY c.cpf, p.nome_completo, p.email, p.telefone
17 HAVING COUNT(*) FILTER (WHERE a.status = 'falta') >= %s
18 ORDER BY total_faltas DESC
```

11.2.2 Explicação

HAVING vs WHERE:

- WHERE: Filtra ANTES de agrupar
- HAVING: Filtra DEPOIS de agrupar
- Usado para filtrar por resultados de agregação

Exemplo:

```
1 HAVING COUNT(*) FILTER (WHERE status = 'falta') >= 3
```

Retorna apenas clientes com 3+ faltas.

Múltiplos FILTER:

- Conta faltas separadamente de atendimentos
- Permite comparar proporção falta/comparecimento
- MAX com FILTER traz data da última falta

11.2.3 Uso

Similar ao de clientes inativos, mas com template de email diferente alertando sobre o histórico de faltas.

12 Consultas de Horários e Disponibilidade

12.1 Verificação de Conflito de Horário

12.1.1 Algoritmo de Detecção de Conflito

Conceito de sobreposição de intervalos:

Dois intervalos de tempo se sobrepõem se:

```
1 # Intervalo A: [inicio_A, fim_A]
2 # Intervalo B: [inicio_B, fim_B]
3
4 # Ha sobreposicao se:
5 inicio_A < fim_B AND fim_A > inicio_B
```

Exemplo visual:

```
Agendamento existente: |-----| (10:00 - 10:30)
Novo agendamento:      |-----| (10:20 - 10:50)
                        ^^^^ conflito
```

12.1.2 Implementação

Passo 1: Buscar agendamentos existentes

```
1 SELECT data_hora_agendamento, duracao_estimada_min
2 FROM Agendamento
3 WHERE barbeiro_id = %s
4 AND DATE(data_hora_agendamento) = %s
5 AND status IN ('pendente', 'confirmado')
```

Passo 2: Para cada agendamento, verificar sobreposição

```
1 novo_inicio = datetime(2025, 1, 15, 10, 20) # 10:20
2 novo_fim = novo_inicio + timedelta(minutes=30) # 10:50
3
4 for ag in agendamentos_existentes:
5     ag_inicio = ag.data_hora
6     ag_fim = ag_inicio + timedelta(minutes=ag.duracao)
7
8     # Verifica sobreposicao
9     if novo_inicio < ag_fim and novo_fim > ag_inicio:
10         return {"tem_conflito": True, "agendamento": ag}
11
12 return {"tem_conflito": False}
```

12.1.3 Uso no Agendamento

Antes de criar agendamento:

1. Cliente escolhe barbeiro, data, horário e serviço
2. Sistema verifica conflito

3. Se houver conflito: Exibe erro com detalhes
4. Se não houver: Permite confirmar

Mensagem de erro informativa:

"Horário indisponível. Já existe agendamento para Maria Santos das 10:00 às 10:30."

13 Boas Práticas e Otimizações

13.1 Uso de Índices

Índices importantes criados:

```
1 CREATE INDEX idx_agendamento_barbeiro
2   ON Agendamento(barbeiro_id, data_hora_agendamento);
3
4 CREATE INDEX idx_agendamento_cliente
5   ON Agendamento(client_id, data_hora_agendamento);
6
7 CREATE INDEX idx_avaliacao_agendamento
8   ON Avaliacao(id_agen);
9
10 CREATE INDEX idx_reserva_cliente
11   ON Reserva(id_cliente, status);
```

Benefícios:

- Acelera buscas por barbeiro/cliente
- Melhora performance em filtros por data
- Otimiza JOINS com tabelas grandes

13.2 Evitar N+1 Queries

Problema N+1:

```
1 # MAU - N+1 queries
2 agendamentos = buscar_agendamentos() # 1 query
3 for ag in agendamentos:
4     servico = buscar_servico(ag.id_servico) # N queries
5     barbeiro = buscar_barbeiro(ag.cpf_barbeiro) # N queries
```

Solução - JOIN:

```
1 # BOM - 1 query
2 SELECT a.*, s.*, b.*
3 FROM Agendamento a
4 JOIN Servico s ON ...
5 JOIN Barbeiro b ON ...
```

13.3 COALESCE para Valores Padrão

Uso consistente:

```
1 -- Evita NULL em agregacoes
2 COALESCE(SUM(preco), 0)
3 COALESCE(AVG(nota), 0)
4 COALESCE(COUNT(*), 0)
```

13.4 Paginação Eficiente

Sempre usar LIMIT e OFFSET:

```
1 SELECT * FROM Cliente
2 ORDER BY nome
3 LIMIT 10 OFFSET 20  -- Página 3 (itens 21-30)
```

Cálculo de offset:

```
1 offset = (pagina - 1) * por_pagina
```

14 Conclusão

Este documento apresentou as principais consultas SQL utilizadas no sistema de gerenciamento de barbearia, explicando:

1. Estrutura e propósito de cada consulta
2. Conexão entre SQL, rotas da API e frontend
3. Conceitos importantes: COALESCE, FILTER, CASE, CTEs
4. Técnicas de otimização e boas práticas
5. Fluxos completos de uso no sistema

As consultas foram projetadas para:

- Serem eficientes e escaláveis
- Fornecerem dados completos em menos requisições
- Facilitarem a manutenção do código
- Permitirem análises detalhadas do negócio